ПРАКТИЧЕСКОЕ ЗАДАНИЕ №3

КЛАССИЧЕСКИЙ И НЕЙРОСЕТЕВОЙ МЕТОД

ПОИСКА КЛЮЧЕВЫХ ТОЧЕК

1 ОСНОВНЫЕ ТЕОРЕТИЧЕСКИЕ ПОЛОЖЕНИЯ

1.1 ЦЕЛЬ РАБОТЫ

Познакомиться с классическим методом поиска ключевых точек и их соответствий, освоить применение SuperPoint для поиска ключевых точек и проанализировать полученные в ходе выполнения работы результаты.

1.2 Что такое ключевые точки

Ключевые точки — это характерные области изображения, хорошо различимые и устойчивые к изменениям масштаба и освещения. Окрестности ключевых точек уникальны, и служат опорой для анализа, сопоставления и восстановления структуры изображения.

1.3 Зачем нужны ключевые точки

Поиск ключевых точек и их сопоставление с рядом изображений является начальным этапом перехода от двухмерных задач к трехмерным. При помощи данного метода решаются задачи моделирования трехмерного объекта, определение положения объекта в пространстве и вычисление расстояние до него.



Рисунок 1. Полученная модель при помощи метода 3D – сканирования с использованием Polycam

Практическое применение данного метода можно найти в таких сферах, как:

Беспилотные автомобили и транспорт:

Понимание сцены (дистанция до машин, пешеходов, разметки);

Помощь в торможении, парковке, удержании полосы.

Дополненная и виртуальная реальность (AR/VR):

Реалистичное совмещение виртуальных объектов с реальной сценой;

Окклюзия (когда виртуальный объект должен скрываться за реальным).

Медицина и анализ полученных в ходе обследований изображений:

Сегментация и измерение объёмов органов (полученных, к примеру при помощи магнитно-резонансной томографии).

1.4 Классический метод поиска ключевых точек

Классический метод поиска ключевых точек делиться на следующие этапы:

1. Детекция ключевых точек и определение их дескрипторов (описание ключевых точек);
2. Сравнения их дескрипторов;
3. Построения линий смещения на изображении.

Алгоритмы детекции ключевых точек называются детекторами. Наиболее часто применяемый из них - детектор углов Харриса (Harris Corner Detector). Данный алгоритм исследует локальные участки изображения и выделяет точки, где интенсивность яркости при смещении участка изменяется в любом направлении. Математическая формулировка описанного определения:

$$E(u,v) = \sum_{x,y} w(x,y)(I(x-u,y+v) - I(x,y))^2, \quad (1)$$

Где:

$E(u,v)$ – функция изменения локального участка при смещении (u – вдоль оси x, v – вдоль оси y);

$w(x,y)$ – весовая функция (бинарное окно или функция Гаусса);

$(I(x-u,y+v) - I(x,y))^2$ – квадрат разницы яркости между смещенным изображением и исходным.

Помимо детектора Харриса, широкую известность приобрел детектор FAST (Features from Accelerated Segment Test). В ходе работы данного детектора производится проверка яркости последовательно расположенных локальных участков вокруг предполагаемой ключевой точки и производится их фильтрация, что обеспечивает его высокую скорость обработки информации.

После обнаружения ключевых точек вычисляются дескрипторы (вектора признаков, описывающие локальную область изображения вокруг ключевых точек). Дескрипторы подразделяются на гистограммные и бинарные.

Гистограммные дескрипторы основываются на направлении градиента яркости в окрестности точки и формируют вектор из вещественных чисел (зачастую являющихся нормализованными). Примеры гистограммных дескрипторов:

SIFT (Scale-Invariant Feature Transform) — один из первых дескрипторов, устойчивых к изменению масштаба;

SURF (Speeded-Up Robust Features) — упрощённая и ускоренная версия SIFT.

Бинарный дескриптор использует попарное сравнения интенсивности пикселей. Результатом являются бинарные строки (последовательности, состоящие из нулей и единиц), что ускоряет сопоставление точек. Примеры бинарного дескриптора:

BRIEF (Binary Robust Independent Elementary Features) — использует случайные пары пикселей для бинарных сравнений;

BRISK/FREAK — используют круговую(кольцевую) структуру выборки вокруг ключевой точки.

1.5 Нейросетевой подход SuperPoint

SuperPoint — современный нейросетевой алгоритм, предназначенный для одновременного обнаружения (детектирования) и описание (дескрипции) ключевых точек на изображении. В отличие от SIFT, BRISK и других алгоритмов, которые основаны на ручных эвристиках, обучение SuperPoint производилось с использованием методов глубокого обучения.

Основным элементов сети является кодировщик (Encoder) — набор сверточных слоев, который извлекает из входного изображения высокоуровневые признаки, служащие основой для последующего детектирования ключевых точек и построения их дескрипторов.

Далее архитектура разделяется на два блока из детектора (Detector head) и дескриптора (Descriptor head). Обе эти части работают на одних и тех же признаках, вырабатываемых кодировщиком. При детектировании генерируется карта вероятности, показывающая, насколько каждый пиксель на изображении является ключевой точкой. При описание формируется векторы признаков изображения по каждой ключевой точки.

2 ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ И ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Практическое задание 1. Получение панорамы

2.1.1 Исходные данные

Скрипт Module_4a/panorama.py склеивает несколько снимков одной сцены в панораму. Углы ищутся детекторами OpenCV (Shi-Tomasi, Харриса или FAST), дескриптор у всех — ORB. Соседние кадры сопоставляются, проецируются на цилиндр и плавно смешиваются.

Снимите самостоятельно:

4 фотографии одной сцены (комната, улица, двор), снятые слева направо по порядку;

камеру поворачивайте на месте вокруг вертикальной оси, не делая шагов в сторону;

соседние кадры должны перекрываться на 30–50 %;

в зоне перекрытия должны быть текстурные детали (мебель, книги, рамки, окна): на однотонной стене точки не найдутся;

не меняйте зум и экспозицию между кадрами, избегайте движущихся объектов;

положите снимки в отдельную папку, например data/panorama; имена файлов должны идти по порядку съёмки (порядок задаётся сортировкой имён). Поддерживаются форматы jpg и png.

2.1.2 Как запустить

Запуск выполняется из корня репозитория:

```
python Module_4a/panorama.py --detector gftt

python Module_4a/panorama.py --input data/my_room --detector harris

python Module_4a/panorama.py --detector orb --scale 1
```

Флаги запуска:

--input — папка с исходными снимками (по умолчанию data/panorama);

--out — папка результата (по умолчанию <input>/out);

--detector — детектор углов: gftt (Shi-Tomasi, по умолчанию), harris (Харрис), orb (FAST);

--max-kp — максимальное число ключевых точек на кадр (по умолчанию 3000);

--scale — масштаб входных кадров (по умолчанию 0.5, 1 — полный размер);

--projection — cylindrical (по умолчанию) или plane. Плоская гомография при широком повороте раздувает холст до десятков тысяч пикселей, и скрипт остановится с сообщением; для поворотной съёмки используйте цилиндр;

--focal — фокусное расстояние в долях ширины кадра (по умолчанию 0.8, для телефона обычно 0.75–0.85);

--show — показывать промежуточные изображения в окнах.

В папке out/<детектор>/ рядом с исходными снимками появятся файлы:

kp_*.jpg — найденные ключевые точки на каждом кадре;

matches_*.jpg — совпадения соседних кадров, прошедшие RANSAC;

panorama.jpg — итоговая панорама;

panorama_borders.jpg — та же панорама с цветными границами исходных кадров;

panorama_plane.jpg — плоская развёртка цилиндра, где прямые линии сцены снова прямые (только для --projection cylindrical).

2.1.3 Отчетность

panorama.jpg, panorama_borders.jpg и panorama_plane.jpg для детектора gftt;

таблицу из вывода консоли для детекторов gftt, harris и orb: число точек на кадр, число совпадений и инлайеров для каждой пары;

вывод: какой детектор дал наиболее надёжную склейку, на какой паре кадров совпадений меньше всего и почему, откуда берётся бочкообразная форма кадров на цилиндре.

Пример отчета приведён ниже.

Пример отчёта

Исходные данные. Сняты четыре кадра одной комнаты, камера поворачивалась на месте слева направо: левое окно, стеллаж со столом, центральное окно, правая стена со столом и рамками. Файлы IMG_20261107_093111.jpg, IMG_20261107_093116.jpg, IMG_20261107_093118.jpg и IMG_20261107_093120.jpg, размер каждого 1920×1440. Соседние кадры перекрываются. Между вторым и третьим кадром перекрытие приходится в основном на штору.

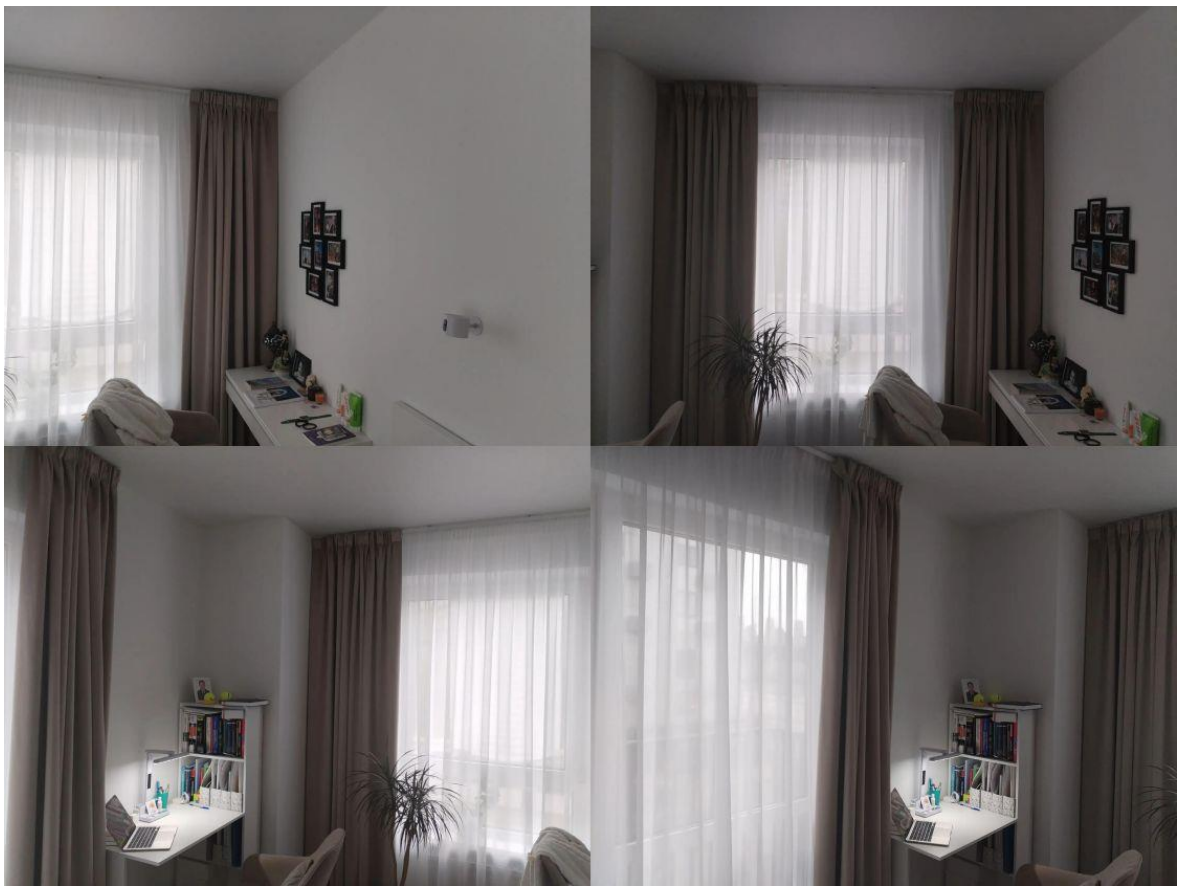


Рисунок. Четыре исходных кадра в порядке съёмки.

Запуск из корня репозитория, по одному прогону на каждый детектор:

```
python Module_4a/panorama.py --detector gftt
python Module_4a/panorama.py --detector harris
python Module_4a/panorama.py --detector orb
```

Ниже приведены результаты детектора Shi-Tomasi (gftt) из папки data/panorama/out/gftt/.

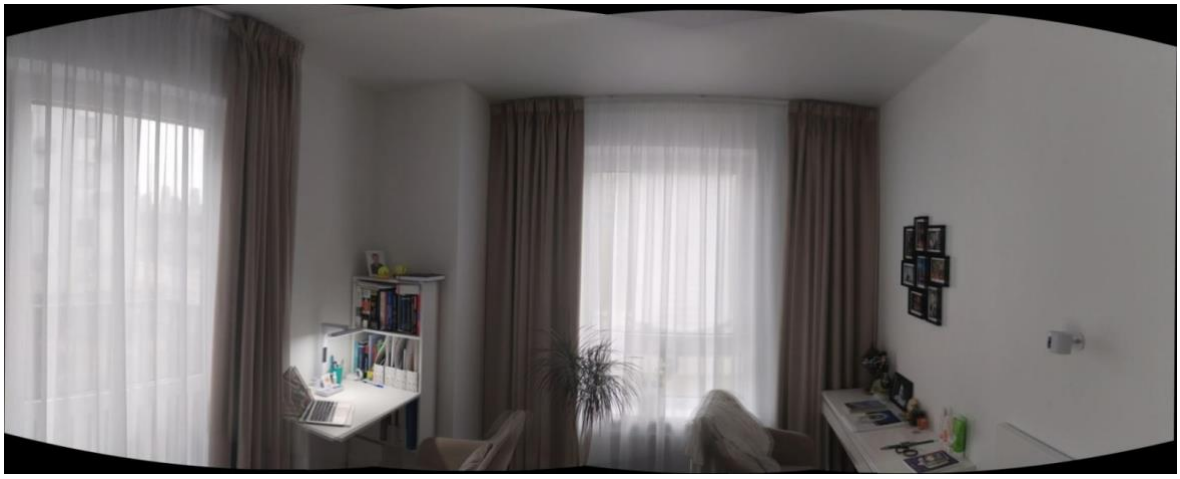


Рисунок. panorama.jpg — цилиндрическая панорама, детектор gftt.

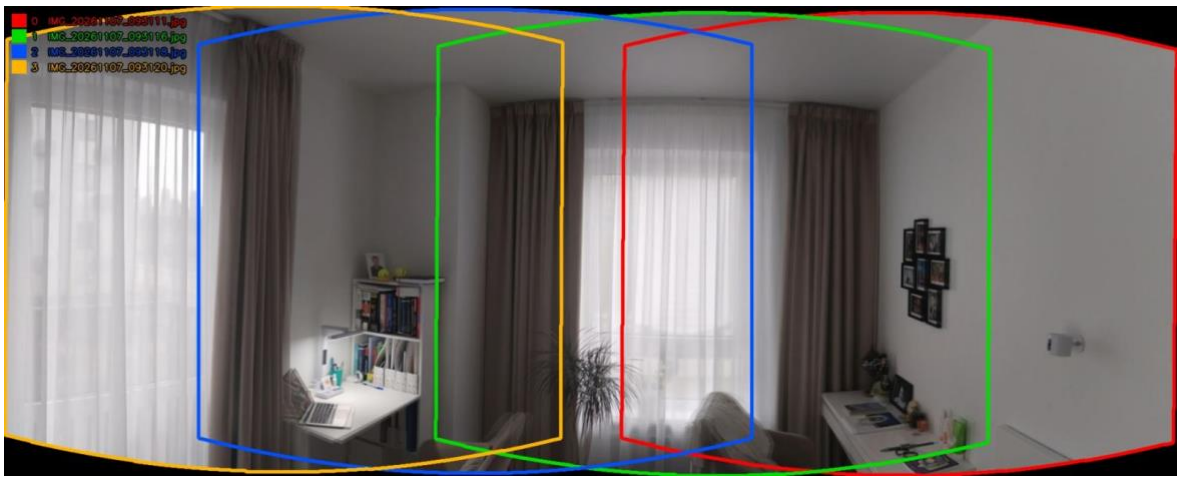


Рисунок. panorama_borders.jpg — границы исходных кадров: красный, зелёный, синий и оранжевый по порядку файлов.

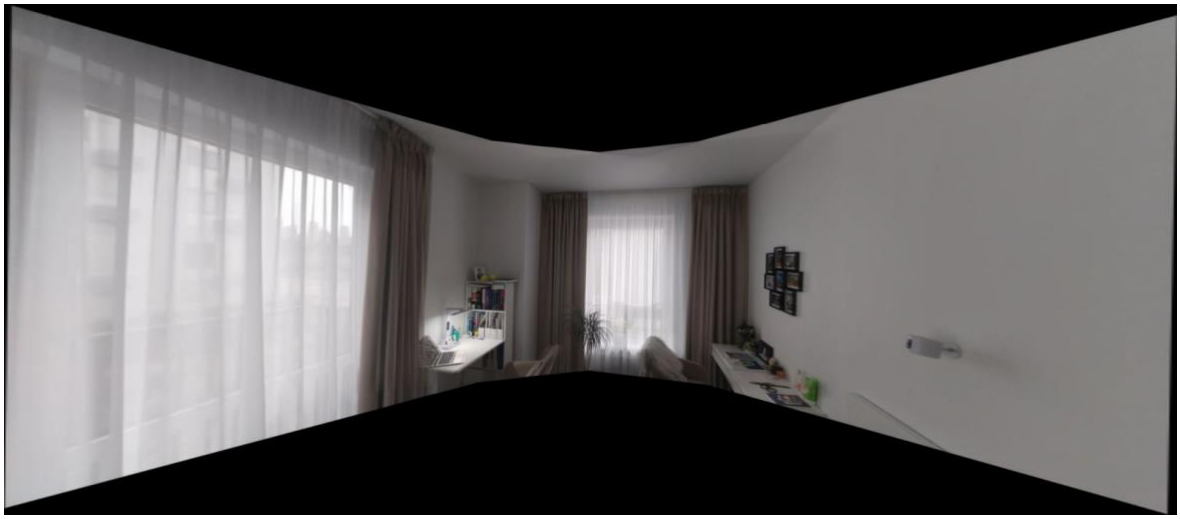


Рисунок. panorama_plane.jpg — та же панорама, развёрнутая на плоскость: карниз и рамы прямые.

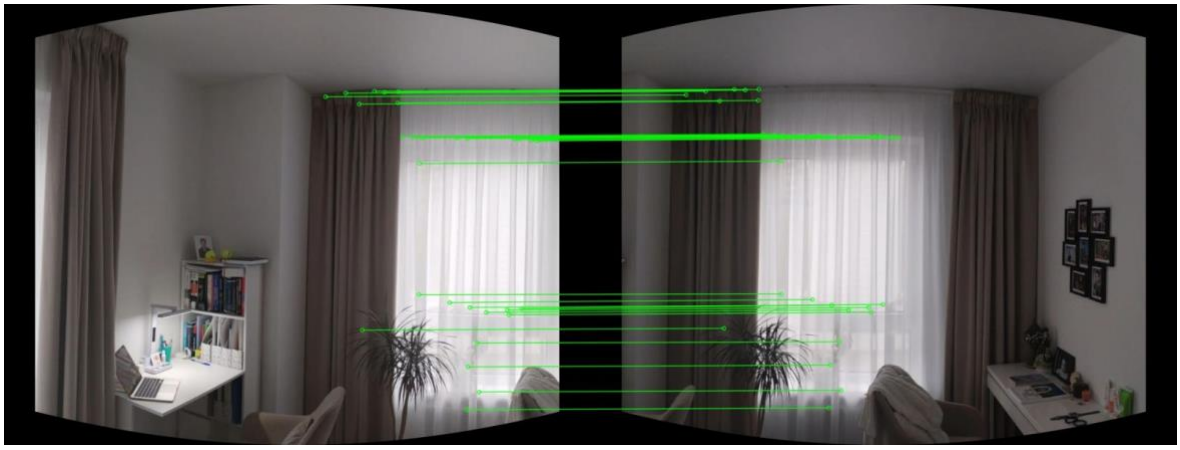


Рисунок. matches_1_2.jpg — совпадения самой слабой пары кадров, прошедшие RANSAC.

Число точек и совпадений из вывода консоли (--scale 0.5, проекция cylindrical):

Детектор	Точки 0	Точки 1	Точки 2	Точки 3	Пара 0–1	Пара 1–2	Пара 2–3
gftt	360	560	583	464	188 / 92	52 / 29	164 / 125

harris	118	199	192	126	88 / 46	10 / 3	77 / 66
orb	2698	2954	2964	2904	491 / 293	73 / 42	585 / 457

В столбцах пар первое число — совпадения дескрипторов, второе — инлайеры RANSAC.

Вывод. Склейка получается у всех трёх детекторов, но устойчивее всего она у ORB и Shi-Tomasi. У ORB на парах 0–1 и 2–3 сотни инлайеров (293 и 457), у gftt — 92 и 125. Детектор Харриса на паре 1–2 оставляет только 3 инлайера: по двум-трём точкам сдвиг оценивается ненадёжно, хотя на этих кадрах панорама всё же собралась.

Меньше всего совпадений у всех детекторов на паре кадров 1–2: у gftt 52 совпадения и 29 инлайеров, у Харриса 10 и 3, у ORB 73 и 42. Перекрытие этих кадров приходится на штору и засвеченное окно, устойчивых углов там мало. Пары 0–1 и 2–3 держатся на стеллаже, рамах и кромках мебели.

Бочкообразная форма кадров на panorama.jpg — это цилиндрическая проекция, а не ошибка совмещения. Снимок переносится на цилиндр радиуса f = 0.8 ширины кадра. В центре угол θ равен нулю и высота сохраняется, к краю θ растёт и та же полоска кадра занимает высоту в cos θ раз меньше, поэтому верх и низ выгибаются дугой. На panorama_plane.jpg панорама развёрнута обратно на плоскость, оптическая ось смотрит в середину дуги (край около ±68°). Карниз и рамы окон снова прямые, а края кадра растянуты: точка на краю считается как f·tan θ, и tan 68° ≈ 2,5.

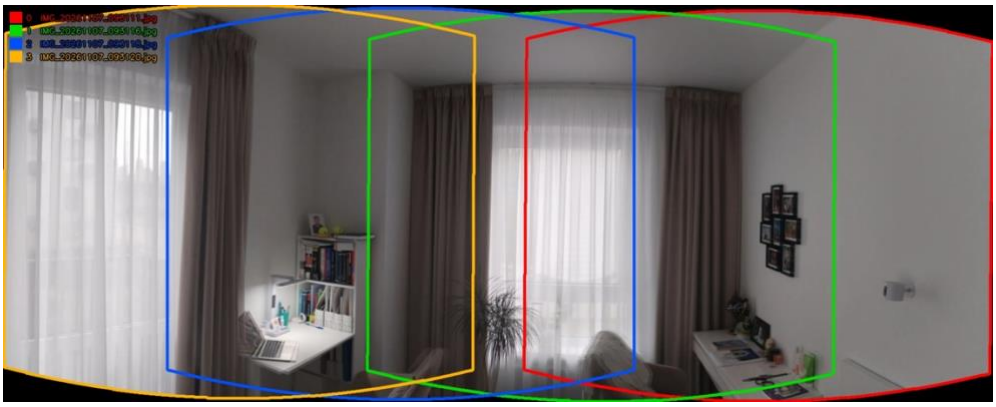


Рисунок 2. Пример панорамы с использованием цилиндрической проекции

2.2 Практическое задание 2. Сравнение ключевых точек двух видео: SuperPoint и SIFT

2.2.1 Исходные данные

Скрипт Module_4b/superpoint_video_match.py сравнивает ключевые точки двух видео одного объекта. Для каждой пары синхронных кадров точки ищутся двумя методами:

нейросетью SuperPoint (классы SuperPointFrontend и PointTracker.nn_match_two_way из репозитория magicleap) и классическим SIFT. Выбросы отсекаются RANSAC-гомографией.

Снимите самостоятельно:

2 коротких видео (6–10 секунд) одного и того же объекта;

во втором видео немного измените точку обзора: сместитесь в сторону или снимите под другим углом. Два одинаковых файла давать нельзя: все точки совпадут сами с собой, и сравнение потеряет смысл;

объект должен быть текстурным и занимать заметную часть кадра;

конвертируйте видео в .avi без звука (Приложение 1).

2.2.2 Зависимости

- `opencv-python`, `numpy`, `torch`: `pip install opencv-python numpy torch`;
- репозиторий SuperPoint с файлом весов `superpoint_v1.pth`, клонированный в папку `Module_4b`:

```
cd Module_4b

git clone https://github.com/magicleap/SuperPointPretrainedNetwork.git
```

2.2.3 Как запустить

Запуск выполняется из корня репозитория:

```
python Module_4b/superpoint_video_match.py \

    --video1 data/my_obj/video1.avi --video2 data/my_obj/video2.avi

python Module_4b/superpoint_video_match.py --video1 ... --video2 ... --
show
```

Флаги запуска:

--video1, --video2 — пути к двум видео (по умолчанию примеры из data/SIFT);

--offset — сдвиг второго видео в кадрах; по умолчанию 0, видео идут синхронно;

--step — обрабатывать каждый n-й кадр (по умолчанию 1);

--width — ширина кадра для обработки (по умолчанию 640);

--conf-thresh — порог уверенности точки SuperPoint (по умолчанию 0.015);

--nn-thresh — порог расстояния дескрипторов при сопоставлении SuperPoint (по умолчанию 0.7);

--max-kp — лимит точек SIFT (по умолчанию 2000);

--weights — путь к весам SuperPoint (по умолчанию Module_4b/SuperPointPretrainedNetwork/superpoint_v1.pth);

--cuda — считать SuperPoint на видеокарте;

--out — папка результата (по умолчанию <папка video1>/out);

--show — показывать сравнение в окне, выход по клавише q.

В папке out рядом с первым видео появятся файлы:

superpoint_vs_sift.mp4 — видео сравнения: сверху SuperPoint, снизу SIFT, зелёные линии — совпадения, прошедшие RANSAC;

superpoint_vs_sift_first.jpg — первый кадр сравнения;

superpoint_vs_sift.csv — статистика по каждому кадру: число точек, совпадений, инлайеров и доля инлайеров для обоих методов.

В конце консоль печатает средние значения по всем кадрам.

2.2.4 Отчетность

Скриншоты из видео superpoint_vs_sift.mp4 и изображение superpoint_vs_sift.csv

вывод: какой метод находит больше точек, у какого совпадения чище (выше доля инлайеров), как распределены точки по кадру и на каких участках сцены каждый метод ошибается.

Пример отчёта

Исходные данные. В папке data/SIFT лежат get.197.025.left.avi и get.197.026.left.avi: по 50 кадров, 960×600, съёмка дороги с трамвая, в кадре попутные автомобили. Файлы побайтово совпадают, поэтому при запуске по умолчанию (--offset 0) каждый кадр сравнивается сам с собой и доля инлайеров равна 1,00 у обоих методов. Чтобы показать реальное смещение сцены, кадр i первого ролика сопоставлен с кадром i+5 второго.

Запуск из корня репозитория:

```
python Module_4b/superpoint_video_match.py --offset 5 --step 5
```

Обработан каждый пятый кадр. Видео сравнения, первый кадр и таблица по кадрам записаны в папку out: superpoint_vs_sift.mp4, superpoint_vs_sift_first.jpg, superpoint_vs_sift.csv.

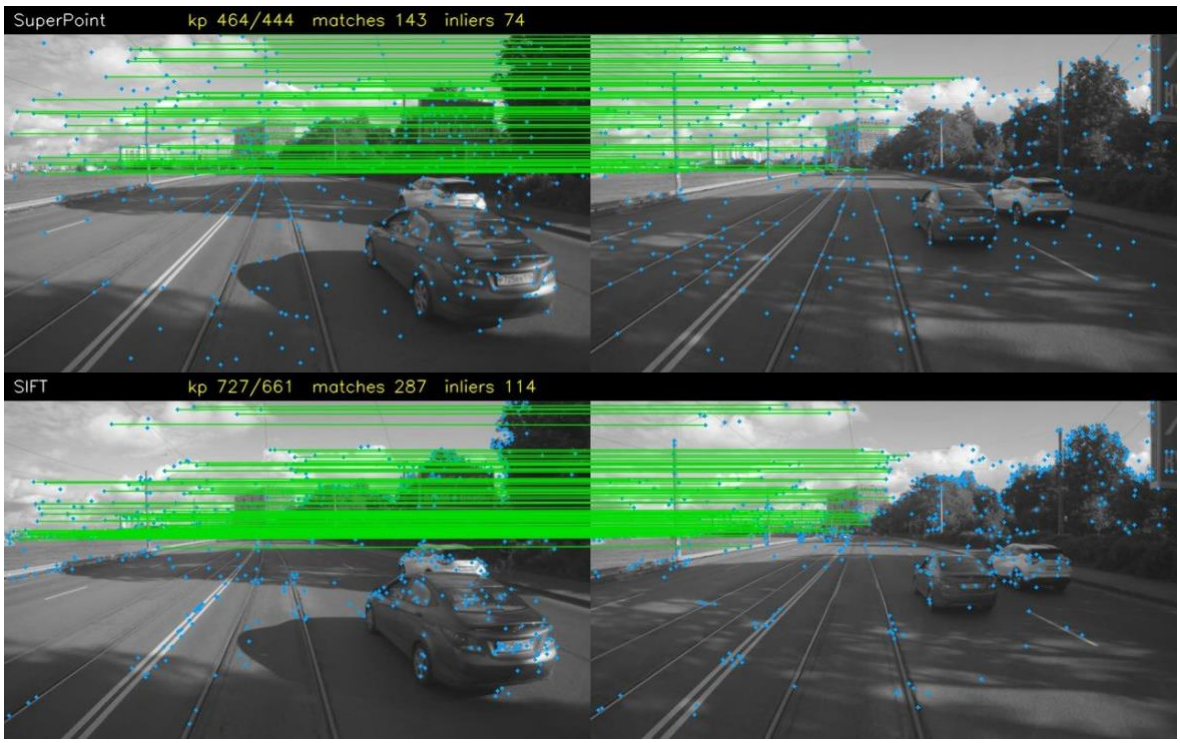


Рисунок. Первый кадр сравнения, сдвиг 5 кадров. Сверху SuperPoint, снизу SIFT. Зелёные линии — совпадения, прошедшие RANSAC; автомобили на правом кадре уже сместились.

Средние значения по обработанным кадрам (ширина кадра 640, порог SuperPoint --nn-thresh 0.7):

Метод	**Точек на кадр**	**Совпадений**	**Инлайеров**	**Доля инлайеров**
SuperPoint	434	152	93	0,61
SIFT	719	292	115	0,40

Вывод. SIFT находит больше точек (около 720 против 430) и больше абсолютных совпадений. Совпадения SuperPoint чище: 61 % проходят RANSAC против 40 % у SIFT. Точки SuperPoint распределены по кадру равномернее — дорога, машины, фасады. У SIFT точки сгущаются на деревьях и кромках облаков, и там же больше ложных соответствий, которые RANSAC отбрасывает. На однотонном асфальте и на бликующих кузовах оба метода теряют точки. Машины, проехавшие за 5 кадров, дают часть соответствий, которые в модель плоскости дороги не входят и в инлайеры не попадают.

Литература

Документация по OpenCV https://opencv.org/

Учебное пособие по калибровке камеры в OpenCVhttps://waksoft.susu.ru/2020/02/29/kalibrovka-kamery-s-ispolzovaniem-s-opencv/

Репозиторий и документация SuperPointhttps://github.com/magicleap/SuperPointPretrainedNetwork

PyTorch Hub — MiDaS (оценка глубины) https://pytorch.org/hub/intelisl_midas_v2/

Руководство по работе с MiDaS от Intelhttps://github.com/isl-org/MiDaS

Обзор детекторов и дескрипторов в OpenCV (SIFT, ORB, FAST, Harris) https://docs.opencv.org/4.x/dc/dc3/tutorial_py_matcher.html

Шпаргалка по компьютерному зрению с OpenCV и Pythonhttps://tproger.ru/translations/opencv-python-guide

Модель для оценки глубины

https://huggingface.co/spaces/depth-anything/Depth-Anything-V2